UADE BEATS

Iteración 4

Revisión de patrones de diseño, documentación UML y verificación

Informe explicativo para presentar en clase

Idea principal. Esta iteración mejora la organización del código y la forma de explicar el sistema. Se separa una regla de visibilidad mediante Strategy, se revisan otros patrones y se completa la documentación con evidencia verificable.

89 tipos	19 diagramas	103 comprobaciones
Documentados a partir de 84 archivos Java.	Clases, casos de uso y actividades, en PNG y SVG.	Resultados registrados en la validación de la iteración.

Resumen de la entrega

El proyecto conserva la funcionalidad de la iteración 4: cuentas con tres roles, eventos en vivo, interfaces de consola y Swing, persistencia SQLite y acceso remoto mediante una API HTTP. El cambio de diseño se concentra en hacer explícita la política que determina qué eventos corresponden a cada cuenta.

El informe diferencia los patrones que ya existían, la mejora implementada y las alternativas que se evaluaron pero no se incorporaron. También presenta los diagramas, las pruebas y los límites del trabajo.

Grupo: 9
Versión: iteración 4 beta
Formato: A4 vertical
Base documental: informes y código del proyecto.

1. Punto de partida y método de trabajo

¿Qué existía antes?

UADE Beats administra el acceso de usuarios a eventos en vivo. La aplicación permite registrar cuentas y artistas, crear eventos, controlar ingresos y conservar la trazabilidad de los intentos. Los permisos distinguen a usuario, artista y administrador.

Base de iteración 4	Trabajo de iteración 4
Java, consola y Swing.	Revisión de responsabilidades y colaboración entre objetos.
SQLite, API HTTP y cliente remoto.	Pruebas sobre los servicios compartidos y documentación reproducible.
Reglas y permisos por rol.	Extracción Strategy para el alcance de eventos.
Suscripción Google / Apple.	Identificación y explicación del Strategy ya existente.

Cómo se organizó con GSD

El trabajo se desarrolló en cuatro pasos: **investigar** el código y las referencias; **planificar** el alcance y sus criterios de aceptación; **ejecutar** la refactorización y la documentación; y **verificar** el resultado con pruebas, cobertura de tipos e inspección visual.

Los planes y la evidencia permiten relacionar cada decisión con un problema concreto. Para esta adaptación a PDF se mantuvo ese criterio y se respetó la indicación de trabajar sin Git.

Arquitectura conservada



Esquema didáctico: las realizaciones apuntan al contrato. Geometría revisada.

Las vistas dependen de **ServicioEventos**. El gestor local coordina los casos de uso; el gestor remoto actúa como Proxy y solicita la ejecución al servidor. Esto evita duplicar las reglas de negocio en cada interfaz.

2. Strategy: lo existente y lo implementado

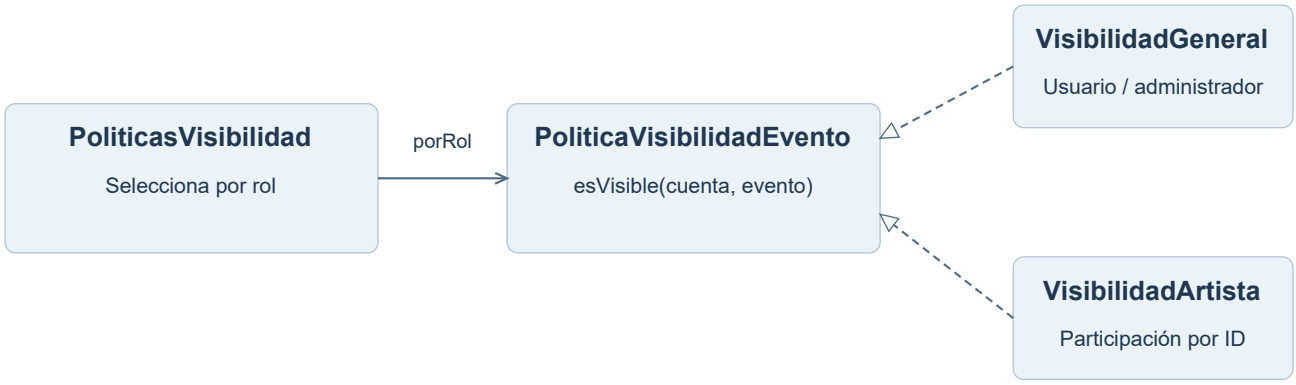
Strategy que ya existía: calendarios

El contrato **VinculoCalendario** define la operación de suscribir un evento. **VinculoGoogle** y **VinculoApple** resuelven esa tarea de forma distinta: una abre un enlace de Google; la otra abre Calendar o utiliza el protocolo webcal. La vista elige el proveedor y el servicio invoca el objeto recibido.

Esta colaboración ya era Strategy: un comportamiento común admite algoritmos intercambiables. La estrategia puede recibirse como parámetro, sin que deba permanecer guardada dentro del contexto. En modo remoto, la apertura ocurre en el equipo cliente.

Strategy nuevo: visibilidad de eventos

Antes, un método de **GestorEventosEnVivo** comprobaba el rol y recorría los artistas asociados al evento. La regla era válida, pero estaba dentro de una clase que también coordina cuentas, accesos y persistencia. Se extrajo para expresar y probar esa variación por separado.



Esquema didáctico: las realizaciones apuntan al contrato. Geometría revisada.

Participante	Responsabilidad
PoliticaVisibilidadEvento	Contrato que responde si una cuenta puede ver un evento.
VisibilidadGeneral	Da alcance al catálogo a usuario y administrador autenticados.
VisibilidadArtista	Comprueba participación por el ID del artista.
PoliticasVisibilidad	Selecciona la política correspondiente al rol.

Delegación en el gestor:

```
return visibilidad.esVisible(actual, evento);
```

El beneficio es separar responsabilidades y facilitar pruebas. No se afirma una mejora de rendimiento ni un cambio en los permisos.

3. Ejemplo y decisiones sobre otros patrones

Ejemplo del comportamiento que se conserva

Si el artista 10 participa en el evento 100, pero no en el 101, la política debe producir estos resultados:

Cuenta	Evento 100	Evento 101
Usuario autenticado	Visible	Visible
Administrador autenticado	Visible	Visible
Artista 10	Visible	No visible
Otra instancia del artista 10, mismo ID	Visible	No visible
Sin cuenta	No visible	No visible

Comparar por ID permite reconocer al artista aunque los objetos se hayan reconstruido desde SQLite. La selección del algoritmo se hace por rol. La política determina alcance; **Permisos** y **ControlSesion** siguen autorizando las acciones. Ver un evento no implica poder modificarlo.

Facade: se identificó un diseño ya existente

GestorEventosEnVivo funciona como Facade: ofrece casos de uso y coordina ControlSesion, ControlAcceso, las entidades, la persistencia y los calendarios. ServicioEventos es su contrato de uso. No se agregó una segunda fachada porque solo duplicaría reenvíos sin simplificar el acceso al subsistema.

Composite: uso real y aplicación futura

La interfaz Swing aprovecha un árbol de componentes: los paneles pueden contener botones, tablas y otros paneles. Esa estructura utiliza el diseño Composite de la biblioteca. En el dominio no hay una jerarquía recursiva de eventos; una lista de eventos no basta para justificar el patrón. Sería razonable si se pidieran festivales con escenarios y subeventos anidados.

Decorator: evaluado y pospuesto

No se encontró una familia propia de envoltorios que agregue responsabilidades manteniendo el mismo contrato. Podría servir para incorporar métricas de persistencia o de notificaciones cuando exista esa necesidad. Agregar capas sin un requisito real aumentaría la complejidad. GestorRemoto es Proxy por su intención de acceso remoto; no se lo reclasificó como Decorator.

4. Documentación UML y casos de uso

Qué se completó en los diagramas

El relevamiento cubre **84 fuentes Java y 89 tipos declarados con nombre**. El atlas integral los organiza en fichas canónicas C001 a C089, con miembros completos y relaciones salientes. Las referencias azules identifican la ficha del tipo relacionado; no agregan clases nuevas.

Las composiciones se justifican por propiedad del ciclo de vida. Un artista existe independientemente de los eventos: esa referencia es una asociación. Las cajas, textos y conectores de las 19 vistas se verifican geométricamente; las relaciones y miembros se contrastan con el inventario del código.

Entregable	Contenido
Clases	Un diagrama integral y siete vistas detalladas por módulo.
Patrones	Una vista de colaboración de Strategy y Facade.
Casos de uso	Tres vistas por actor y una vista integrada.
Actividades	Seis flujos: tres por rol y tres transversales.

Responsabilidades de los tres actores

Actor	Casos de uso y alcance
Usuario	Consulta eventos y capacidad; solicita ingreso a su nombre; ve sus registros y suscribe calendarios.
Artista	Crea y gestiona eventos donde participa; consulta sus conectados y suscribe sus eventos a calendarios.
Administrador	Administra cuentas y eventos; activa, desactiva y expulsa usuarios; consulta los registros generales.

Las **20 acciones** se obtuvieron de Permisos.java compilado. Los objetivos agrupan operaciones afines; abrir Swing permanece en la matriz técnica. El administrador no se modela como especialización del actor usuario: no dispone de la acción de consultar sus propios registros.

> El inventario automático controla cobertura. La interpretación de asociaciones, multiplicidades y propiedad requiere además revisar la semántica del código.

5. Flujos principales explicados

Solicitud de ingreso a un evento

El ingreso sigue un orden de validación. Ese orden permite explicar por qué un intento se acepta o se rechaza y dónde se conserva su trazabilidad.

Paso	Responsabilidad y resultado
1. Permiso y cuenta propia	El gestor exige la acción. El usuario normal no puede actuar a nombre ajeno.
2. Existencia	Se buscan usuario y evento antes de continuar con el acceso.
3. Autorización y plan	AccesoEvento valida actividad, habilitación y cobertura de suscripción.
4. Estado y horario	El recital debe estar en curso y dentro de la ventana de ingreso.
5. Capacidad y duplicado	Debe quedar lugar y el usuario no debe estar conectado previamente.
6. Registro y guardado	Se genera el resultado, se actualizan los datos y se guarda en SQLite.

Una falta de permiso se rechaza antes de procesar el ingreso; no equivale a un intento de acceso rechazado por el recital. Si faltan usuario o evento, se genera un registro con una referencia de reemplazo en memoria, pero el repositorio lo omite al guardar por sus claves foráneas.

Gestión de eventos por el artista

El artista selecciona un evento propio. El gestor verifica permiso y pertenencia; el evento valida la transición de estado. Solo entonces se modifica, se guarda y se refresca la vista. Si el evento es ajeno, la operación se deniega. La política de alcance se reutiliza también en este control.

Desactivación de usuarios por el administrador

Se comprueba el permiso y se impide desactivar la propia cuenta. Al desactivar a otro usuario se lo desconecta de los eventos y se registran las expulsiones antes de guardar. Este caso muestra que un cambio de cuenta también puede afectar conexiones y trazabilidad.

6. Verificación y límites del resultado

Evidencia registrada de la iteración

La revisión volvió a ejecutar **103 comprobaciones de la aplicación** y nueve pruebas del detector de superposiciones. Se verificaron las 19 vistas contra su modelo geométrico y se inspeccionaron los diagramas y los detalles de clases a escala legible.

Área	Comprobación
Políticas y permisos	Roles, identidad por ID, referencias nulas, eventos propios y ajenos y matriz completa de acciones.
Servicio local y HTTP	Listados, obtención de URL de calendario y cambios de estado, incluidos rechazos para eventos ajenos.
SQLite	Guardar y cargar conserva asociaciones y visibilidad del artista, usando una base temporal.
Diagramas	89 tipos con miembros, 20 acciones trazadas y 19 pares PNG/SVG con render verificado.

Qué no demuestra esta evidencia

No se confirmó una suscripción real en Google o Apple ni se automatizaron todos los clics de Swing. La inspección de los diagramas y la cobertura de tipos tampoco equivalen a una certificación formal de conformidad UML o a una auditoría de seguridad integral.

Mejoras futuras identificadas

Quedaron señaladas la caducidad de sesiones, una validación de datos más completa y la comprobación de la URL pública. También conviene revisar la consistencia entre memoria y base cuando falla un guardado: un rollback SQL no restaura automáticamente los objetos que ya fueron modificados.

Conclusión de validación. La refactorización se verificó sobre los caminos relevantes de la regla extraída. Los límites y problemas heredados se documentaron sin atribuirles una corrección que esta iteración no implementó.

7. Conclusiones y material para la defensa

El aporte principal de la iteración es una regla de alcance explícita, acompañada de documentación trazable y pruebas para defender las decisiones. La funcionalidad existente mantiene sus contratos de API, DTO y esquema SQLite.

Pregunta para la exposición	Respuesta orientativa
¿Por qué usar Strategy?	Porque existen dos algoritmos de alcance que pueden expresarse y probarse mediante un contrato común.
¿Qué Strategy era previo?	La selección de Google o Apple para suscribir el calendario de un evento.
¿Por qué no implementar todos los patrones?	Porque una abstracción debe resolver una necesidad concreta; agregarla sin ese motivo aumenta el costo de mantenimiento.
¿Cambiaron los permisos?	No. La política de alcance se separó y sus resultados se contrastaron con los controles existentes.

Anexos incorporados al PDF

Se incluye el SVG del diagrama completo de clases de esta entrega. En un lector compatible se accede desde el panel de archivos adjuntos. Algunos visores web no muestran ese panel; los archivos originales también se conservan en la entrega del proyecto.

Referencias

OMG: especificación UML 2.5.1

PlantUML: notación de diagramas de clases

Refactoring.Guru: catálogo de patrones de diseño

Java SE 21: Component y Container

Fuentes internas: PATRONES.md, REVISION-UML.md, CASOS-DE-USO.md y VERIFICACION.md, en docs/iteracion-4/, más los contratos y las políticas del código Java.

Idea para cerrar la exposición: la calidad del diseño se sostiene en la justificación de sus decisiones y en la evidencia del comportamiento, no en la cantidad de patrones utilizados.